Trabajo Práctico: Modelos de Control de Acceso (RBAC vs. ABAC)

Equipo 01 - Integrantes

- Tomas Ruiz
- Facundo Sorasio
- Mateo Sparta
- Rodrigo Vargas
- Jose Zerpa

TP 3: Modelos de Control de Acceso

Consigna

Analice un escenario donde un médico solo puede ver historias clínicas en horario de oficina y desde la red del hospital. Compare por qué el modelo ABAC es superior al RBAC en este caso, considerando atributos de entorno y sujeto en tiempo de ejecución.

Análisis y Resolución

Para abordar el análisis del caso evaluamos la transición de un esquema de permisos estático hacia uno capaz de procesar variables contextuales en tiempo real, comparando los dos enfoques.

Paso 1: Evaluación del Modelo RBAC (Control de Acceso Basado en Roles)

En esta primera instancia, el acceso se gestiona mediante la pertenencia a un grupo (Rol: Médico). Al aplicar las restricciones de la consigna notamos las siguientes limitaciones:

- Falta de contexto: El sistema valida la identidad del médico pero ignora si la conexión proviene de una red externa o si se realiza fuera de horario.
- Explosión de roles: Para cumplir con la consigna, nos veríamos obligados a crear roles excesivamente específicos (por ejemplo: "Médico_Turno_Mañana_Red_Interna"), lo cual es muy difícil de escalar y auditar.

Paso 2: Implementación y Ventajas del Modelo ABAC (Control de Acceso Basado en Atributos)

Al migrar hacia un control basado en atributos el sistema deja de otorgar permisos de forma binaria y pasa a evaluar reglas lógicas. A continuación, comparamos cómo operan ambos modelos frente a los atributos de control:

Criterio de Evaluación	RBAC (Estático)	ABAC (Dinámico)
Variable de Decisión	Pertenencia a un grupo.	Atributos del sujeto, entorno y recurso.
Validación de Horario/Red	No contemplada de forma nativa.	Requisito obligatorio para el acceso.
Flexibilidad	Baja, requiere cambios estructurales.	Alta, permite ajustar políticas sin crear roles.

Para garantizar el acceso seguro, ABAC ejecuta una política lógica similar a la siguiente:

1) Se define una tabla para almacenar historias clínicas de todos los pacientes y una RLS(Row Level Security) para definir una política que filtrará qué filas se pueden ver basándose en las condiciones:

```
CREATE TABLE historias_clinicas (
id SERIAL PRIMARY KEY,
paciente_nombre TEXT,
diagnostico TEXT,
medico_asignado TEXT);


ALTER TABLE historias_clinicas ENABLE ROW LEVEL SECURITY;
```

2) Se crea una politica ABAC para querys del tipo SELECT:

```
CREATE POLICY politica_acceso_medicos_estricta
ON historias_clinicas
FOR SELECT
TO medicos_role
USING (
      (current_time >= '08:00:00'
      AND current_time <= '18:00:00')
      AND (inet_client_addr() <<= '192.168.1.0/24')
      AND (medico_asignado = current_user)
);
```

En el ejemplo anterior se utilizó una base de datos PostgreSQL, en la cual también se puede hacer uso de sentencias GRANT/REVOKE para dar o quitar permisos sobre tablas, vistas o esquemas, pero que pertenecen al modelo RBAC ya que no son suficientes para cumplir con una regla de negocio que dependa del contexto(solo mira el rol).

Para profundizar mas en la sintaxis de estas politicas, se muestra una politica para INSERT:

```
CREATE POLICY politica_insert_medicos ON historias_clinicas
FOR INSERT
WITH CHECK
      (current_time >= '08:00:00'
      AND current_time <=  '18:00:00')
      AND (inet_client_addr() <<= '192.168.1.0/24')
);
```

Donde es importante notar el cambio que se hace entre “USING”(que se aplica a las filas que ya están en la tabla, por lo tanto va en querys de tipo SELECT, UPDATE y DELETE)y “WITH CHECK”(se aplica en filas que van entrando o cambiando en la tabla, en querys como INSERT y UPDATE)

Clasificación de los atributos evaluados por ABAC

En el modelo ABAC, la decisión de acceso se basa en la evaluación de distintos atributos en tiempo de ejecución. En este escenario intervienen tres categorías principales:

Atributos del sujeto

Corresponden a las características del usuario que solicita acceso.

Rol del usuario: Médico (*medicos_role*)
Usuario autenticado (*current_user*)
Relación con el recurso: el médico debe coincidir con el valor almacenado en *medico_asignado*

Atributos del entorno

Representan condiciones contextuales existentes al momento de la solicitud.

Hora actual del sistema (*current_time*)
Dirección IP del cliente (*inet_client_addr()*)
Red autorizada del hospital (*192.168.1.0/24*)

Atributos del recurso

Son propiedades del objeto al que se intenta acceder.

La historia clínica como recurso protegido
Campo *medico_asignado*, que indica qué profesional tiene asignado ese registro

Evaluación de la política

El acceso a la historia clínica solo se concede cuando se cumplen simultáneamente las siguientes condiciones:

El usuario pertenece al rol de médico.
La solicitud se realiza entre las 08:00 y las 18:00.
La conexión proviene de la red interna del hospital.
El médico autenticado coincide con el médico asignado al registro.

Seguridad

La superioridad de ABAC en este escenario se fundamenta en tres pilares que mejoran la integridad del sistema:

- Seguro por Contexto: Incluso si las credenciales de un médico son comprometidas, el atacante no puede acceder a las historias clínicas si intenta ingresar desde una red externa o fuera del horario establecido.
- Menor Privilegio Dinámico: El permiso de acceso solo existe durante el tiempo y en el lugar que la tarea lo requiere, reduciendo la superficie de ataque.
- Auditoría Granular: Al basarse en atributos el sistema registra no solo quién entró, sino desde qué IP y en qué momento exacto, haciendo que el rastreo de anomalías sea mucho más claro.